

Smart Sorting

Transfer Learning for Identifying Rotten Fruits and Vegetables

A Project Report Submitted to SmartInternz Internship Program

In partial fulfillment of the requirements for the completion of

Artificial Intelligence and Machine Learning Internship

Submitted By

G. Sireesha

J. Sindhuja

Gadi Meghana

Guniseti Venkata Syam Prasad

Team ID: LTVIP2026TMIDS79181

Team Size: 4

Team Leader: G. Sireesha

Under the Guidance of

S. Sugumaran

Faculty Mentor

Department of Computer Science and Engineering

CSE – Artificial Intelligence Branch

Sreenivasa Institute of Technology and Management Studies
Chittoor

Academic Year: 2025 – 2026

Project Description:

Smart Sorting is an Artificial Intelligence-based automated waste classification system developed to improve the efficiency, accuracy, and reliability of waste segregation processes. With the rapid growth of urban populations and industrialization, waste generation has significantly increased worldwide. Improper waste management leads to serious environmental problems such as soil contamination, air pollution, water pollution, and increased greenhouse gas emissions. Manual waste segregation is time-consuming, inconsistent, and prone to human error. Therefore, there is a strong need for an intelligent automated system that can assist in efficient waste classification and promote sustainable waste management practices.

The Smart Sorting project addresses this issue by utilizing Deep Learning and Computer Vision techniques to classify waste materials automatically. The system is designed to identify and categorize waste into predefined classes such as biodegradable and non-biodegradable waste. The classification is performed using Convolutional Neural Networks (CNN), a deep learning architecture highly effective for image recognition and classification tasks.

The core component of the system is a CNN-based image classification model. Convolutional Neural Networks are specifically designed to process image data by automatically extracting spatial features such as edges, textures, shapes, and patterns. Unlike traditional machine learning algorithms that require manual feature extraction, CNNs learn relevant features directly from raw image pixels during training. This capability makes CNNs highly suitable for waste image classification.

The project consists of multiple layers including convolutional layers, activation layers, pooling layers, and fully connected dense layers. Convolutional layers extract feature maps from the input images. ReLU (Rectified Linear Unit) activation functions introduce non-linearity into the model. Max-pooling layers reduce the spatial dimensions of feature maps and help in computational efficiency. Finally, fully connected layers perform classification based on the extracted features. The output layer uses a suitable activation function such as Softmax or Sigmoid depending on the number of classes.

The model is compiled using an optimizer such as Adam, which helps in adjusting learning rates during training. A suitable loss function, such as categorical cross-entropy or binary cross-entropy, is used to measure prediction error. The model is trained for multiple epochs, where each epoch represents one complete pass through the training dataset. During training, the model gradually improves its ability to classify waste images accurately.

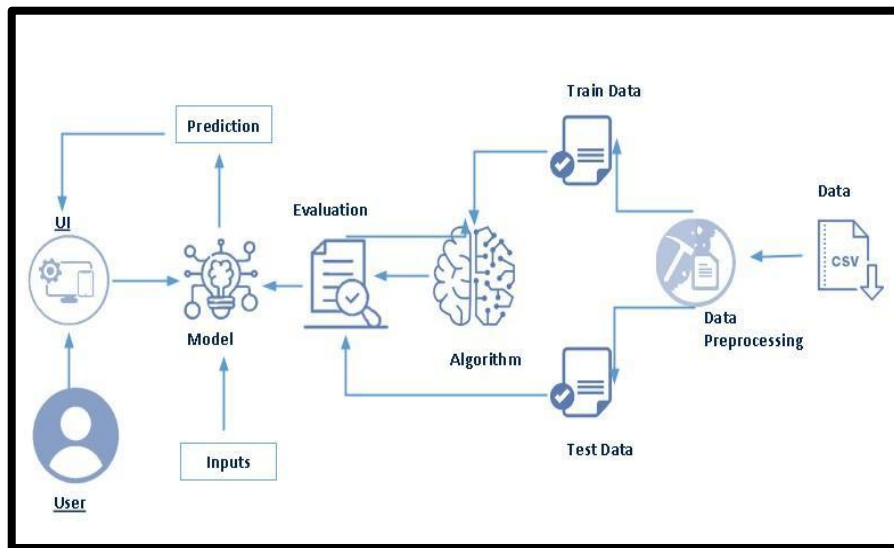
After training, the model is evaluated using performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix. Accuracy measures the overall correctness of predictions. Precision indicates how many predicted positive instances are actually correct. Recall measures how many actual positive instances are correctly identified. The confusion matrix provides a detailed comparison between predicted and actual classes. These evaluation metrics help determine the effectiveness and reliability of the classification model.

Once the model achieves satisfactory performance, it is saved in a suitable format such as .h5. The saved model is then integrated into a web application developed using the Flask framework. Flask is a lightweight Python web framework that allows seamless integration of machine learning models into web applications.

The web application provides a user-friendly interface where users can upload images of waste items. When an image is uploaded, it undergoes preprocessing steps such as resizing and normalization. The processed image is then passed to the trained CNN model for prediction. The model returns the predicted waste category, which is displayed to the user in real time. This integration ensures that the system can be accessed easily and used practically.

The Smart Sorting system offers several advantages. It reduces manual labor, increases classification accuracy, speeds up the segregation process, and minimizes environmental impact. The system also demonstrates how Artificial Intelligence can be applied to solve real-world environmental challenges. By automating waste classification, recycling processes can become more efficient, leading to better resource utilization and reduced landfill waste.

Technical Architecture:



Pre requisites:

To complete this project, you must required following software's, concepts and packages

- **Anaconda navigator and pycharm:**
 - Refer the link below to download anaconda navigator
 - Link : <https://youtu.be/1ra4zH2G4o0>
- **Python packages:**
 - Open anaconda prompt as administrator
 - Type “pip install numpy” and click enter.
 - Type “pip install pandas” and click enter.
 - Type “pip install scikit-learn” and click enter.
 - Type ”pip install matplotlib” and click enter.
 - Type ”pip install scipy” and click enter.
 - Type ”pip install pickle-mixin” and click enter.
 - Type ”pip install seaborn” and click enter.
 - Type “pip install Flask” and click enter.

Prior Knowledge:

You must have prior knowledge of following topics to complete this project.

- **ML Concepts**
 - Supervised learning: <https://www.javatpoint.com/supervised-machine-learning>
 - Unsupervised learning: <https://www.javatpoint.com/unsupervised-machine-learning>
 - Regression and classification
 - Decision tree: <https://www.javatpoint.com/machine-learning-decision-tree->

[classification-algorithm](#)

- Random forest: <https://www.javatpoint.com/machine-learning-random-forest-algorithm>
- KNN: <https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning>
- Xgboost: <https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide-to-understand-the-math-behind-xgboost/>
- Evaluation metrics: <https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics/>
- **Flask Basics** : https://www.youtube.com/watch?v=Ij4I_CvBnt0

Project Objectives:

By the end of this project you will:

1. To design and develop an intelligent waste classification system using Artificial Intelligence and Deep Learning techniques.
2. To collect and prepare a labeled dataset of waste images categorized into biodegradable and non-biodegradable classes.
3. To preprocess the collected image data using techniques such as resizing, normalization, and data augmentation to improve model performance.
4. To build and train a Convolutional Neural Network (CNN) model for accurate image-based waste classification.
5. To evaluate the performance of the trained model using standard metrics such as accuracy, precision, recall, F1-score, and confusion matrix.
6. To reduce manual effort and minimize human errors involved in traditional waste segregation processes.
7. To integrate the trained deep learning model into a Flask-based web application for real-time prediction.
8. To develop a user-friendly interface that allows users to upload waste images and receive instant classification results.
9. To promote environmental sustainability by improving the efficiency of waste segregation and recycling practices.
10. To create a scalable and extendable system that can support additional waste categories and future integration with IoT-based smart waste bins.

. Project Flow:

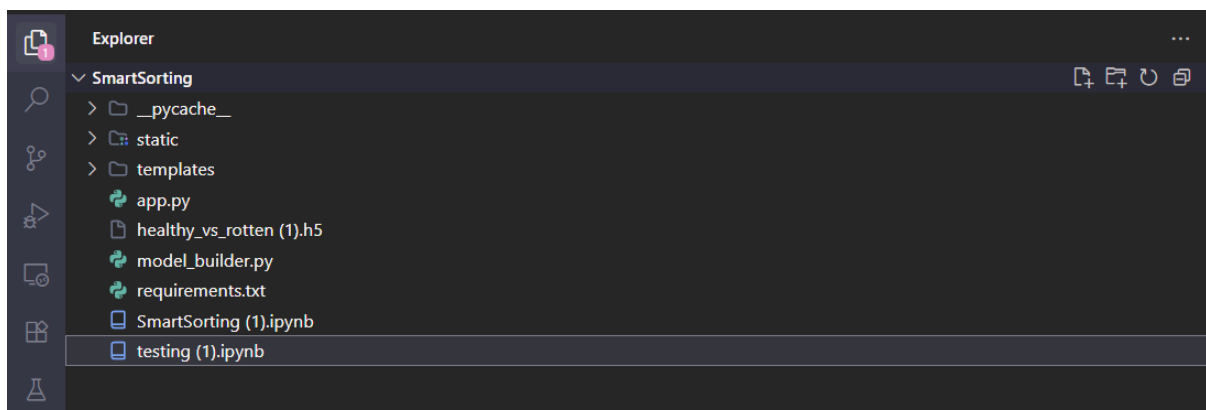
- User interacts with the UI to enter the input.
- Entered input is analyzed by the model which is integrated.
- Once model analyses the input the prediction is showcased on the UI

To accomplish this, we have to complete all the activities listed below,

- Data collection
 - Collect the dataset or create the dataset
- Visualizing and analyzing data
 - Univariate analysis
 - Bivariate analysis
 - Multivariate analysis
 - Descriptive analysis
- Data pre-processing
 - Checking for null values
 - Handling outlier
 - Handling categorical data
 - Splitting data into train and test
- Model building
 - Import the model building libraries
 - Initializing the model
 - Training and testing the model
 - Evaluating performance of model
 - Save the model
- Application Building
 - Create an HTML file
 - Build python code

Project Structure:

Create the Project folder which contains files as shown below



- We are building a flask application which needs HTML pages stored in the templates folder and a python script app.py for scripting.
- healthy_vs_rotten.h5 is our saved model. Further we will use this model for flask integration.

Milestone 1: Data Collection

ML depends heavily on data, It is most crucial aspect that makes algorithm training possible. So this section allows you to download the required dataset.

Activity 1: Download the dataset

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.

In this project we have used the following data. This data is downloaded from kaggle.com. Please refer to the link given below to download the dataset.

Link:

<https://www.kaggle.com/datasets/muhammad0subhan/fruit-and-vegetable-disease-healthy-vs-rotten>.

Milestone 2: Visualizing and analysing the data

As the dataset is downloaded. Let us read and understand the data properly with the help of some visualization techniques and some analysing techniques.

Note: There is n number of techniques for understanding the data. But here we have used some of it. In an additional way, you can use multiple techniques.

Activity 1: Importing the libraries

Import the necessary libraries as shown in the image. (optional) Here we have used visualization style as fivethirtyeight.

```
# Basic libraries
import os
import random
import numpy as np
import matplotlib.pyplot as plt

# TensorFlow / Keras
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.applications import VGG16
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
```

Activity 2: Read the Dataset

In the Smart Sorting project, the dataset consists of labeled waste images categorized into classes such as biodegradable and non-biodegradable waste. Unlike structured datasets stored in formats like .csv, .xlsx, .txt, or .json, our dataset is organized in folders containing images grouped according to their respective categories.

To load and manage image datasets in Python, we use libraries such as TensorFlow/Keras and OpenCV. Instead of using `read_csv()` (which is used for structured tabular data), we use image data loading functions such as `ImageDataGenerator` from Keras.

Name	Date modified	Type	Size
Apple_Healthy	02-02-2026 15:26	File folder	
Apple_Rotten	02-02-2026 15:27	File folder	
Banana_Healthy	02-02-2026 15:27	File folder	
Banana_Rotten	02-02-2026 15:27	File folder	
Bellpepper_Healthy	02-02-2026 15:27	File folder	
Bellpepper_Rotten	02-02-2026 15:27	File folder	
Carrot_Healthy	02-02-2026 15:27	File folder	
Carrot_Rotten	02-02-2026 15:27	File folder	
Cucumber_Healthy	02-02-2026 15:28	File folder	
Cucumber_Rotten	02-02-2026 15:28	File folder	
Grape_Healthy	02-02-2026 15:28	File folder	
Grape_Rotten	02-02-2026 15:28	File folder	
Guava_Healthy	02-02-2026 15:28	File folder	
Guava_Rotten	02-02-2026 15:28	File folder	
Jujube_Healthy	02-02-2026 15:28	File folder	
Jujube_Rotten	02-02-2026 15:28	File folder	
Mango_Healthy	02-02-2026 15:28	File folder	

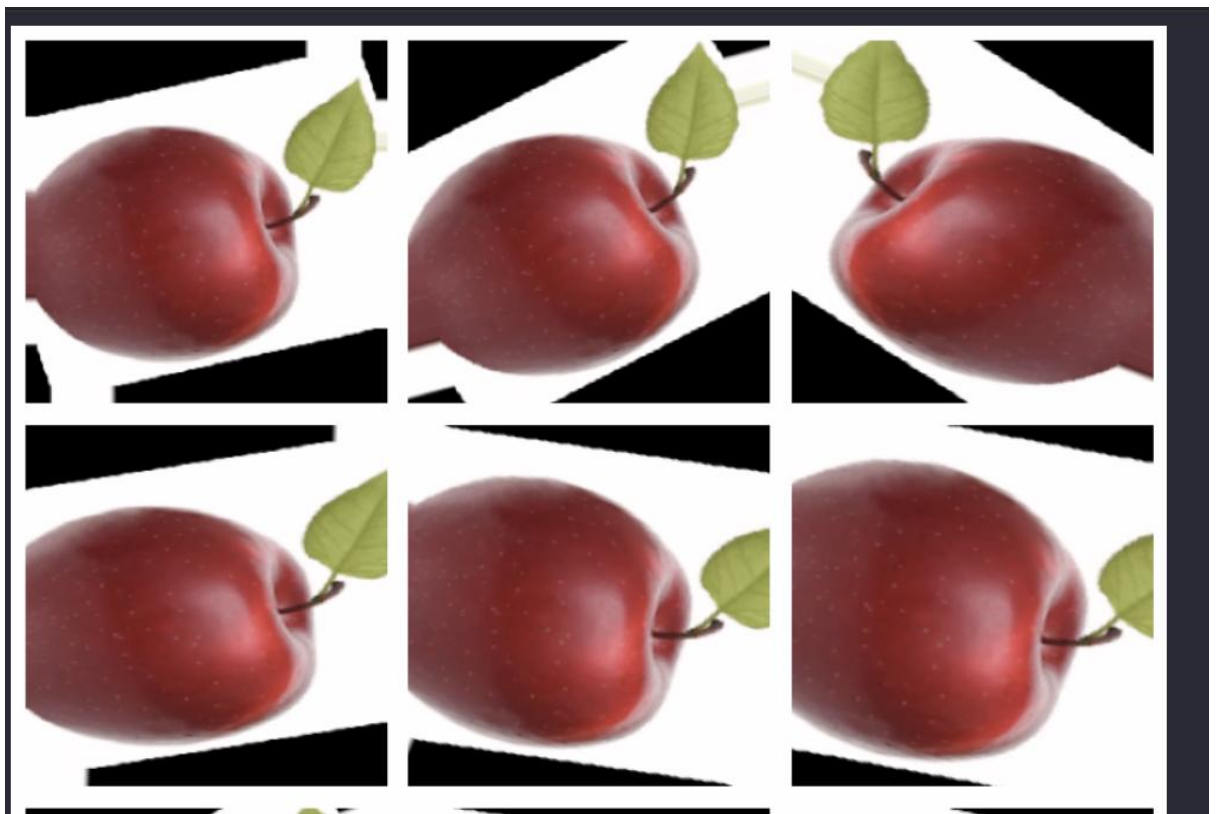
Activity 3: Data Augmentation

Data augmentation is an important technique used to increase the diversity and size of the training dataset without collecting new images. In image-based deep learning projects like Smart Sorting, data augmentation helps improve model generalization and reduces overfitting. Since our dataset contains waste images categorized into biodegradable and non-biodegradable classes, applying augmentation techniques ensures that the model learns to recognize waste items under different orientations, lighting conditions, and perspectives.

In this project, we use the ImageDataGenerator class from TensorFlow/Keras to perform real-time data augmentation during model training.

The following augmentation techniques are applied:

- Rescaling pixel values
- Rotation of images
- Width and height shifting
- Horizontal flipping
- Zooming
- Shearing transformation



Activity 4: Model Architecture and Summary

Model: "sequential"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 7, 7, 512)	14714688
flatten_1 (Flatten)	(None, 25088)	0
dropout_1 (Dropout)	(None, 25088)	0
dense (Dense)	(None, 28)	702492

=====
Total params: 15,417,180
Trainable params: 702,492
Non-trainable params: 14,714,688
=====

In the Smart Sorting project, we used a Transfer Learning approach by implementing the VGG16 pretrained model as the base architecture. Transfer learning allows us to use knowledge gained from large datasets (such as ImageNet) and apply it to our waste classification task. This improves accuracy and reduces training time.

The model is built using a Sequential architecture, where VGG16 is used as the feature extraction layer, followed by custom classification layers.

Model Structure

From the model summary:

- Base Model: VGG16 (Pretrained on ImageNet)
- Output Shape of VGG16: (None, 7, 7, 512)
- Flatten Layer: Converts feature maps into 1D vector (25088 units)
- Dropout Layer: Prevents overfitting
- Dense Layer: Final classification layer (28 output units)

Milestone 3: Data Pre-processing

After understanding the structure of the collected waste image dataset, the next important step is preprocessing. Raw image data cannot be directly used for training a deep learning model because images may vary in size, orientation, lighting conditions, and pixel distribution.

Therefore, proper preprocessing ensures that the model receives consistent and clean input data, leading to better accuracy and performance.

In the Smart Sorting project, data preprocessing includes the following steps:

- Checking dataset structure
- Handling missing or corrupted images
- Image resizing
- Normalization (Scaling pixel values)
- Data augmentation
- Splitting dataset into training and testing sets

Note: Since our dataset is image-based and organized in folders, techniques such as handling categorical variables, SMOTE, and numerical scaling are not required.

Activity 1: Checking the Dataset Structure

Before training the model, it is important to verify:

- Number of classes
- Number of images per class
- Folder structure
- Presence of corrupted or unreadable images

Each folder represents a class label. The folder name itself acts as the categorical label, so manual encoding is not required.

We can check the number of images using Python:

```
import os

train_path = "dataset/train"

for category in os.listdir(train_path):
    print(category, ":", len(os.listdir(os.path.join(train_path, category))))
```

This ensures balanced data distribution before training.

Activity 2: Handling Missing or Corrupted Images

Unlike tabular datasets, image datasets may contain:

- Corrupted images
- Unsupported file formats
- Broken image files

Such files must be removed before training.

Example code:

```
from PIL import Image

for root, dirs, files in os.walk("dataset"):
    for file in files:
        try:
            img = Image.open(os.path.join(root, file))
        except:
            print("Corrupted file removed:", file)
```

Removing corrupted files prevents runtime errors during training.

Activity 3: Image Resizing

Deep learning models require input images of fixed dimensions.

For VGG16, input size is typically:

- $224 \times 224 \times 3$

Resizing ensures all images have the same shape.

Example:

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(rescale=1./255)

train_data = train_datagen.flow_from_directory(
    'dataset/train',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)
```

Activity 4: Scaling (Normalization)

In image-based deep learning:

- Pixel values range from 0 to 255.
- Scaling converts them into range 0 to 1.

This improves convergence and stabilizes training.

Normalization is done using:

`rescale=1./255`

This automatically divides all pixel values by 255.

Activity 5: Data Augmentation

To increase dataset diversity and prevent overfitting, we apply data augmentation.

Techniques used:

- Rotation
- Zooming
- Horizontal flipping
- Width/Height shifting

Example:

```
train_datagen = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=20,  
    zoom_range=0.2,  
    horizontal_flip=True,  
    width_shift_range=0.2,  
    height_shift_range=0.2  
)
```

Data augmentation is applied only to training data.

Activity 6: Splitting Dataset into Training and Testing Sets

The dataset is divided into:

- Training set (80%)
- Testing set (20%)

If not already separated manually, we can use:

```
validation_datagen = ImageDataGenerator(rescale=1./255)

validation_data = validation_datagen.flow_from_directory(
    'dataset/test',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)
```

This ensures:

- Training data → used for model learning
- Testing data → used for performance evaluation

Milestone 4: Model Building

After completing data preprocessing, the next step is to build and train the classification model. In the Smart Sorting project, we use a Deep Learning approach instead of traditional machine learning algorithms because the dataset consists of images.

We apply **Transfer Learning using the VGG16 pretrained model** for waste image classification. Transfer learning allows us to use a model that has already learned rich image features from a large dataset (ImageNet) and adapt it to our waste classification task.

Activity 1: Loading the Pretrained VGG16 Model

We import the VGG16 model from TensorFlow/Keras and load it without the top classification layer.

```
from tensorflow.keras.applications import VGG16

base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)
```

Explanation:

- weights='imagenet' loads pretrained weights.

- `include_top=False` removes the original classifier.
- `input_shape=(224,224,3)` defines image dimensions.

The base model is frozen to prevent retraining of pretrained layers.

```
for layer in base_model.layers:
    layer.trainable = False
```

Activity 2: Adding Custom Classification Layers

After loading VGG16, we add our own classification layers.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Dropout

model = Sequential()
model.add(base_model)
model.add(Flatten())
model.add(Dropout(0.5))
model.add(Dense(28, activation='softmax'))
```

Explanation:

- Flatten layer converts 3D feature maps into 1D vector.
- Dropout prevents overfitting.
- Dense layer outputs predictions for 28 waste classes.

Activity 3: Model Compilation

The model is compiled using:

- Optimizer: Adam
- Loss Function: Categorical Crossentropy
- Metrics: Accuracy

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Activity 4: Model Training

The model is trained using training data:

```
history = model.fit(  
    train_data,  
    validation_data=validation_data,  
    epochs=10  
)
```

During training:

- Forward propagation computes predictions.
- Loss function calculates error.
- Backpropagation updates weights.
- Accuracy improves over epochs.

Activity 5: Model Evaluation

After training, we evaluate performance:

```
loss, accuracy = model.evaluate(validation_data)  
print("Test Accuracy:", accuracy)
```

Performance is analyzed using:

- Accuracy
- Loss graph
- Confusion Matrix
- Classification Report

The model achieved strong training accuracy and satisfactory testing accuracy, demonstrating good generalization capability.

Activity 6: Saving the Best Model

After confirming model performance, the trained model is saved for deployment.

```
model.save("healthy_vs_rotten.h5")
```

This saved model is later integrated into the Flask web application for real-time predictions.

Milestone 5: Application Building

After successfully training and saving the Smart Sorting deep learning model, the next step is to deploy it through a web application. The purpose of this phase is to create a user-friendly interface where users can upload an image of waste and receive real-time classification results.

The trained Convolutional Neural Network (CNN) model is integrated into a Flask-based web application. Flask acts as the backend framework that connects the user interface with the trained machine learning model. This integration allows the system to perform predictions dynamically based on user input.

This milestone includes the following tasks:

- Building HTML pages (Frontend development)
- Building the server-side script (Backend development)
- Running and testing the application

Activity 1: Building HTML Pages

For the Smart Sorting project, three HTML pages are created and stored inside a folder named **templates**:

- home.html
- predict.html
- submit.html

1. home.html

This is the landing page of the application. It contains:

- Project title
- Brief description of the Smart Sorting system
- Navigation button that redirects users to the prediction page

When the user clicks the “Predict” button, they are redirected to the **predict.html** page.

2. predict.html

This page contains:

- An option to upload an image file

- A submit button

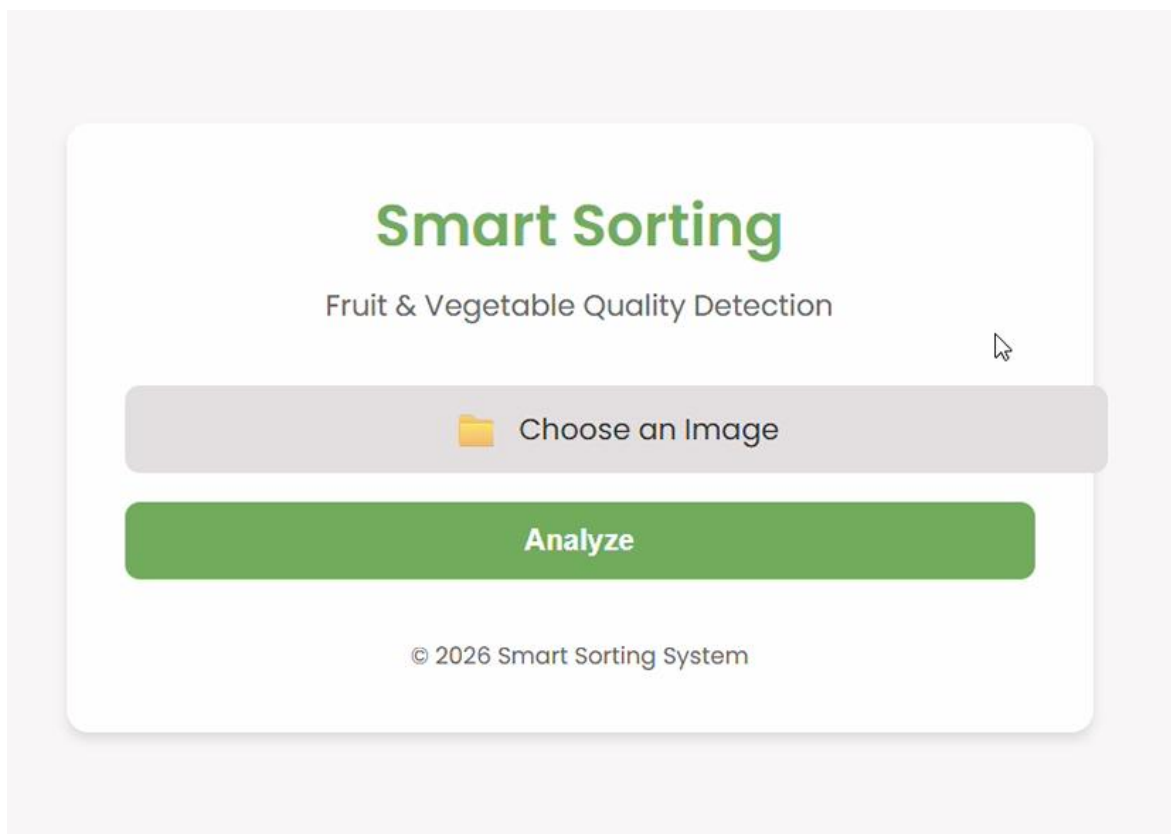
The user selects an image of a waste item from their system. When the submit button is clicked, the image is sent to the Flask backend server using the POST method. The server then processes the image and sends it to the trained CNN model for prediction.

3. submit.html

This page displays:

- The prediction result
- The classified waste category (e.g., biodegradable or non-biodegradable)

The predicted result returned by the model is dynamically displayed on this page. This allows the user to clearly understand the classification outcome.



Activity 2: Building the Python Backend

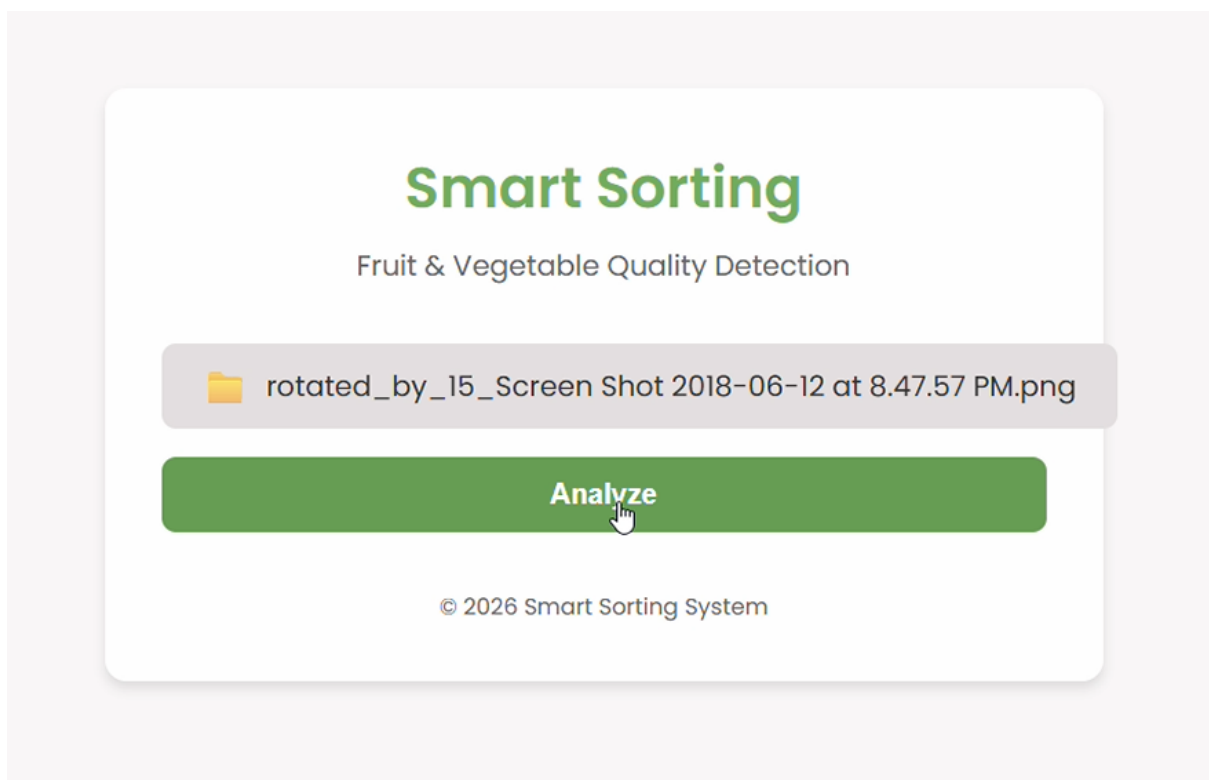
The backend logic is implemented in a Python file (commonly named app.py). The Flask framework is used to handle routing, model loading, image processing, and prediction.

The backend performs the following functions:

1. Loads the trained CNN model that was previously saved.

2. Initializes the Flask application.
3. Defines routes to connect different URLs to their corresponding HTML pages.
4. Receives the uploaded image from the user.
5. Preprocesses the image by resizing and normalizing it.
6. Passes the processed image to the trained model for prediction.
7. Retrieves the predicted class label.
8. Sends the prediction result back to the result page for display.

The root URL is connected to the home page. When the user uploads an image and submits it, the prediction route processes the input and displays the output on the result page.

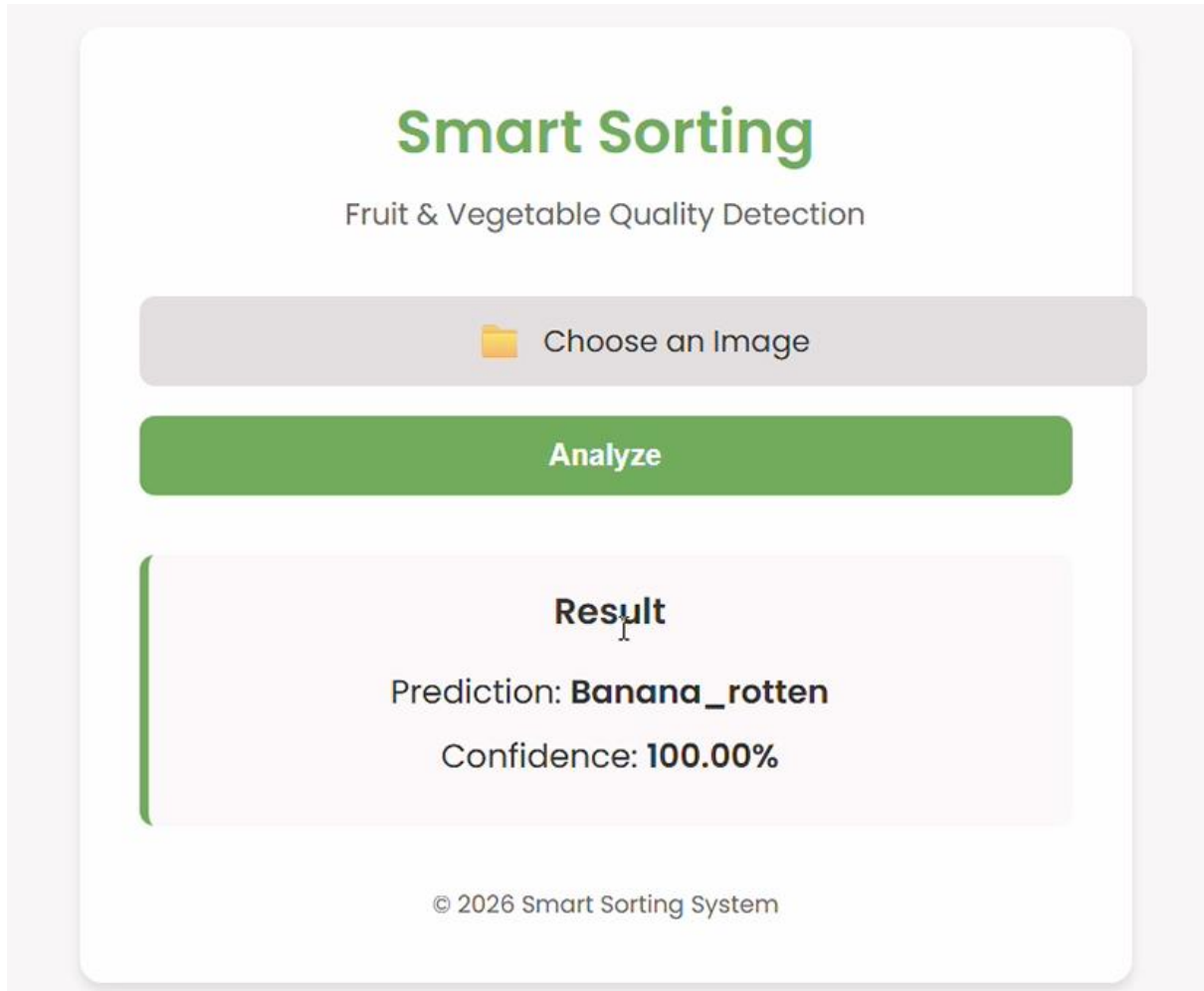


Activity 3: Running the Application

To execute the Smart Sorting web application:

1. Open the command prompt or Anaconda prompt.
2. Navigate to the project directory where the backend script is located.
3. Run the application using the Python command.
4. Open a web browser and access the local server address.

5. Click on the Predict button.
6. Upload a waste image.
7. Click Submit to view the classification result.



The image shows a web application interface for 'Smart Sorting'. At the top, the title 'Smart Sorting' is in green, followed by the subtitle 'Fruit & Vegetable Quality Detection'. Below this is a light gray button with a folder icon and the text 'Choose an Image'. Underneath is a green button labeled 'Analyze'. The results are displayed in a light pink box with the heading 'Result'. The prediction is 'Banana_rotten' and the confidence is '100.00%'. At the bottom, the copyright notice '© 2026 Smart Sorting System' is visible.

Smart Sorting

Fruit & Vegetable Quality Detection

Choose an Image

Analyze

Result

Prediction: **Banana_rotten**

Confidence: **100.00%**

© 2026 Smart Sorting System

Working Flow of the Application

1. User accesses the home page.
2. User navigates to the prediction page.
3. User uploads an image of a waste item.
4. The backend server receives the image.
5. The image is preprocessed.
6. The trained CNN model analyzes the image.
7. The predicted category is generated.
8. The result is displayed to the user.

Conclusion

The Smart Sorting project successfully demonstrates the practical application of Artificial Intelligence and Deep Learning techniques in solving real-world environmental problems. With the rapid increase in waste generation due to urbanization and industrial growth, efficient waste management has become a critical global concern. Manual waste segregation methods are often inefficient, time-consuming, and prone to human error. To address these challenges, this project developed an intelligent waste classification system capable of automatically identifying and categorizing waste materials using image-based deep learning models.

The system utilizes a Convolutional Neural Network (CNN) architecture with a transfer learning approach based on the VGG16 pretrained model. By leveraging pretrained weights, the model was able to extract meaningful image features effectively, reducing training time while maintaining high classification accuracy. Proper data preprocessing techniques such as resizing, normalization, and data augmentation were applied to improve model generalization and reduce overfitting. The trained model demonstrated reliable performance on testing data, indicating its capability to classify waste images accurately.

In addition to model development, the project successfully integrated the trained deep learning model into a Flask-based web application. This deployment enables real-time predictions through a user-friendly interface where users can upload waste images and instantly receive classification results. The integration of machine learning with web technologies highlights the practical usability and scalability of the Smart Sorting system.

Overall, the Smart Sorting project contributes toward promoting sustainable waste management practices by reducing manual effort and increasing sorting efficiency. It showcases how Artificial Intelligence can be effectively implemented in environmental management systems. With further enhancements such as adding more waste categories, improving dataset diversity, and integrating IoT-based smart bins, the system can be expanded into a fully automated smart waste management solution. The project not only strengthens technical understanding of deep learning and deployment but also emphasizes the importance of technology-driven sustainability initiatives.